

Control Flow: Conditional Statements (CodeGrade)

 [_ \(https://github.com/learn-co-curriculum/python-p3-conditional-statements\)](https://github.com/learn-co-curriculum/python-p3-conditional-statements)  [_ \(https://github.com/learn-co-curriculum/python-p3-conditional-statements/issues/new\)](https://github.com/learn-co-curriculum/python-p3-conditional-statements/issues/new)

Learning Goals

- Conditionally execute code with `if/else` statements.
- Handle exceptions using `try/except` statements.
- Use dictionary mapping to handle `switch/case` logic.

Key Vocab

- **Interpreter:** a program that executes other programs. Python programs require the Python interpreter to be installed on your computer so that they can be run.
- **Python Shell:** an interactive interpreter that can be accessed from the command line.
- **Data Type:** a specific kind of data. The Python interpreter uses these types to determine which actions can be performed on different data items.
- **Exception:** a type of error that can be predicted and handled without causing a program to crash.
- **Code Block:** a collection of code that is interpreted together. Python groups code blocks by indentation level.
- **Function:** a named code block that performs a sequence of actions when it is called.
- **Scope:** the area in your program where a specific variable can be called.

Introduction

In the last lesson, we saw how to use comparison methods and logical operators in Python. In this lesson, we'll see more examples of how to use those tools to perform control flow using **conditional statements** with the `if/else` and `try/except` keywords. We will also discuss how Python approaches `switch/case` statements.

Make sure to code along with the Python examples in the Python shell to help get a feel for the syntax.

`if/else` Statements

Python has slightly different syntax for writing conditional statements using `if/else` than JavaScript. Here's a relatively complex `if/else` statement in JavaScript:

```
// JavaScript
let dog = "cuddly";
let owner;

if (dog === "hungry") {
  owner = "Refilling food bowl.";
} else if (dog === "thirsty") {
  owner = "Refilling water bowl.";
} else if (dog === "playful") {
  owner = "Playing tug-of-war.";
} else if (dog === "cuddly") {
  owner = "Snuggling.";
} else {
  owner = "Reading newspaper.";
}
```

Here's how we can write the equivalent statement in Python:

```
# Python
dog = "cuddly"
```

```
if dog == "hungry":
    owner = "Refilling food bowl."
elif dog == "thirsty":
    owner = "Refilling water bowl."
elif dog == "playful":
    owner = "Playing tug-of-war."
elif dog == "cuddly":
    owner = "Snuggling."
else:
    owner = "Reading newspaper."
```

Notice that the `owner` variable here was not defined above like it would be in JavaScript. This is because Python does not require the variable to be declared or initialized prior to the conditional. In Python, a variable can be in scope for the entire class or function. Therefore, we can use the `owner` variable for the rest of the class since it is in **local scope**.

Truthy/Falsy Values

In order to use control flow effectively, it's important to know what values Python treats as "truthy" and "falsy".

As we saw in the lesson on data types, there are many values Python considers falsy:

- Empty lists `[]`
- Empty tuples `()`
- Empty dictionaries `{}`
- Empty sets `set()`
- Empty strings `''` or `""`
- Zero of any numeric type (`0` , `0.0`)
- `None`
- And, of course, `False`

Using those values in control flow means the condition will be **False** :

```
def control_flow(value):  
    if value:  
        # if the value is truthy  
        print("yep!")  
    else:  
        # if the value is falsy  
        print("nope!")
```

```
control_flow(False)  
# "nope!"  
control_flow(None)  
# "nope!"  
control_flow(True)  
# "yep!"  
control_flow("")  
# "nope!"  
control_flow(0)  
# "nope!"  
control_flow("0")  
# "yep!"
```

Conditional Expressions

Python also allows us to use **conditional expressions** (or **ternary operators**) to evaluate the truthiness of complex statements in a single line.

```
age = 1
```

```
is_baby = 'baby' if age < 2 else 'not a baby'
```

This is the equivalent of the following `if/else` statement:

```
age = 1
if age < 2:
    is_baby = 'baby'
else:
    is_baby = 'not a baby'
```

Conditional expressions in Python are always of the format:

```
value_if_true if condition else value_if_false
```

Python requires a default value (preceded by the `else` keyword) in every conditional statement. It may seem like a pain at first, but it helps to prevent unexpected exceptions and `None` s as you continue to build your application.

`try/except` Statements

Throughout our Python assignments so far, we have seen a number of different **Exceptions**. As we learned in our "Error Messages" lesson, Exceptions are a type of error that we can intercept so that our Python application can continue to run. `try/except` statements are the tool that allow us to perform these interceptions.

Let's take a look at how we might handle a common mathematical exception. Copy the following code into the Python shell and try to run the `divide()` function with different arguments.

```
def divide(num1, num2):
    try:
        quotient = num1 / num2
        print(quotient)
    except:
        print("An error occurred")
```

Did you find any arguments that gave you trouble? The `divide()` function will fail to perform its primary task if `num2` is 0 or either of the numbers is of a non-numerical type. Our `try/except` statement allowed our function to run to completion, but `"An error occurred"` is not a particularly helpful message.

Since we know the types of exceptions we might see, let's rewrite our code to be a little more descriptive:

```
def divide(num1, num2):
    try:
        quotient = num1 / num2
        print(quotient)
    except ZeroDivisionError:
        print("Error: num2 cannot be equal to 0")
    except TypeError:
        print("Error: input must be of type int or float")
```

That's looking much more descriptive now!

Finally, let's take a look at `finally`. Copy and paste the following code into the Python shell and test `divide()` with a variety of different arguments:

```
def divide(num1, num2):
    try:
        quotient = num1 / num2
        print(quotient)
    except ZeroDivisionError:
        print("Error: num2 cannot be equal to 0")
    except TypeError:
        print("Error: input must be of type int or float")
    finally:
        print("Isn't division fun?")
```

Use of the `finally` keyword at the end of a `try/except` statement allows us to perform actions that we want to occur regardless of whether or not an exception has been thrown.

NOTE: You might see some unhandled exceptions if you provide `divide()` too many arguments or names that have not been defined. Since these technically occur before `divide()` starts working, they cannot be handled with a `try/except` statement inside of `divide()` .

Dictionary Mapping

Unlike JavaScript, Python does not have `switch/case` statements. Python can handle `switch/case` logic in `if/else` statements, but for very long sets of conditions, it may be worthwhile to use **dictionary mapping** instead.

NOTE: Python 3.10 has introduced `match/case` statements which function very similarly to `switch/case` statements in JavaScript. Though we are using an earlier version of Python in our curriculum, you can explore this new feature in the [Python 3.10 documentation](https://docs.python.org/3/whatsnew/3.10.html#pep-634-structural-pattern-matching).
[🔗_\(https://docs.python.org/3/whatsnew/3.10.html#pep-634-structural-pattern-matching\)](https://docs.python.org/3/whatsnew/3.10.html#pep-634-structural-pattern-matching)

Read through the following JavaScript code:

```
// JavaScript
let dog = "cuddly";
let owner;

switch (dog) {
  case "hungry":
    owner = "Refilling food bowl.";
    break;
  case "thirsty":
    owner = "Refilling water bowl.";
    break;
  case "playful":
    owner = "Playing tug-of-war.";
    break;
  case "cuddly":
    owner = "Snuggling.";
    break;
}
```

```
default:
    owner = "Reading newspaper.";
    break;
}
```

This `switch/case` statement takes the status of the `dog` as a string and sets the state of the owner accordingly.

Let's take a look at how we might do that with an `if/elif/else` statement in Python:

```
# Python
dog = "cuddly"

if dog == "hungry":
    owner = "Refilling food bowl."
elif dog == "thirsty":
    owner = "Refilling water bowl."
elif dog == "playful":
    owner = "Playing tug-of-war."
elif dog == "cuddly":
    owner = "Snuggling."
else:
    owner = "Reading newspaper."
```

As you can see, there is some repeated code in `dog ==`, but the code is still more concise than with a true `switch/case` statement in JavaScript.

Now let's look at how we would handle this with dictionary mapping. Copy and paste the following code into the Python shell:

```
dog = "cuddly"

dict_map = {
    "hungry": "Refilling food bowl.",
    "thirsty": "Refilling water bowl.",
```



```
"playful": "Playing tug-of-war.",  
"cuddly": "Snuggling.",  
}
```

```
# Remember that a dictionary's .get() method lets us set a default value!  
owner = dict_map.get(dog, "Reading newspaper.")
```

This approach is *very* concise, but the mapping dictionary itself is not so intuitive to read; as we can see, the keys describe the state of the `dog` while the values describe the state of the `owner`. Dictionary mapping is a valuable tool for long lists of conditions, but `if/elif/else` statements are typically the preferred method for handling `switch/case` logic in Python.

Instructions

Time to get some practice! Write your code in the `lib` folder's `control_flow.py`. Run `pytest -x` to check your work. Your goal is to practice using control flow in Python to familiarize yourself with the syntax. There is a JavaScript version of the solution for each of these deliverables in the `js/index.js` file you can look at (but if you want an extra challenge, try solving them in Python without looking at the JavaScript solution).

Write a function `admin_login()` that takes two arguments, a username and a password. If the username is "admin" or "ADMIN" and the password is "12345", return "Access granted". Otherwise, return "Access denied".

```
admin_login("sudo", "12345")  
# "Access denied"  
admin_login("admin", "12345")  
# "Access granted"  
admin_login("ADMIN", "12345")  
# "Access granted"
```

NOTE: `and` takes precedence over `or` in Python. Depending on how you write your conditional statement(s), you may need to group pieces together with parentheses `()` so that they are evaluated together.

Write a function `hows_the_weather()` that takes in one argument, a temperature. If the temperature is below 40, return "It's brisk out there!". If the temperature is between 40 and 65, return "It's a little chilly out there!". If the temperature is above 85, return "It's too dang hot out there!". Otherwise, return "It's perfect out there!"

```
hows_the_weather(33)
# "Brisk!"
hows_the_weather(99)
# "Too dang hot"
hows_the_weather(75)
# "Perfect!"
```

Write a function `fizzbuzz()` takes in a number. For multiples of three, return "Fizz" instead of the number. For the multiples of five, return "Buzz". For numbers which are multiples of both three and five, return "FizzBuzz". For all other numbers, just return the number itself.

```
fizzbuzz(1)
# 1
fizzbuzz(2)
# 2
fizzbuzz(3)
# Fizz
fizzbuzz(4)
# 4
fizzbuzz(5)
# Buzz
fizzbuzz(15)
# FizzBuzz
```

Write a function `calculator()` that takes three arguments: an operation and two numbers. If the operation is one of the following: `+`, `-`, `*`, or `/`, return the value of calling the operation on the two numbers. Otherwise, output a message saying "Invalid operation!" and return `None`.

```
calculator("+", 1, 1)
# 2
```

```
calculator("-", 3, 1)
# 2
calculator("*", 3, 2)
# 6
calculator("/", 4, 2)
# 2
calculator("nope", 4, 2)
# "Invalid operation!"
# None
```

Conclusion

Since you're already familiar with these control flow structures from JavaScript, you should have a good intuition of when it's appropriate to use these different tools. Try and develop familiarity with the differences in syntax between JavaScript and Python first, so that you'll be able to take advantage of some of Python's unique features in your own code.

One excellent resource for familiarizing yourself with the syntax and the coding standards for Python developers is the [PEP 8 - Style Guide for Python Code](https://peps.python.org/pep-0008/)  [\(https://peps.python.org/pep-0008/\)](https://peps.python.org/pep-0008/). Make sure to bookmark this resource and refer to it if you're ever unsure how to format a particular block of code.

Resources

- [Python ternary operators](https://book.pythontips.com/en/latest/ternary_operators.html)  [\(https://book.pythontips.com/en/latest/ternary_operators.html\)](https://book.pythontips.com/en/latest/ternary_operators.html)
- [PEP 8 - Style Guide for Python Code](https://peps.python.org/pep-0008/)  [\(https://peps.python.org/pep-0008/\)](https://peps.python.org/pep-0008/)

This tool needs to be loaded in a new browser window

Load Control Flow: Conditional Statements (CodeGrade) in a new window

